

Experiment 1A:

1. Create a DataFrame Question: Create a DataFrame from the following dictionary: python CopyEdit data = { 'Name': ['Alice', 'Bob', 'Charlie', 'David'], 'Age': [25, 30, 35, 40], 'City': ['New York', 'Los Angeles', 'Chicago', 'Houston'] } Display the DataFrame.

CODE:

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}

df = pd.DataFrame(data)

print(df, '\n')
```

2. Filtering Rows Question: From the above DataFrame, filter out the rows where the age is greater than 30

CODE:

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}

fdf = df[df["Age"]>30]

print(fdf, '\n')
```

3.Adding a New Column Question: Add a new column Salary with values [50000, 60000, 70000, 80000] to the existing DataFrame.

CODE:

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}

df['Salary']=[50000, 60000, 70000, 80000]

print(df,'\n')
```

4. Update Column Values Question: Increase the salary of all people by 10%.

CODE:

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}

df['Salary']=df['Salary']*1.1

print(df,'\n')
```

5. Drop a Column Question: Remove the City column from the DataFrame

CODE:

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}

df=df.drop(columns=['City'])
```

```
print(df,'\n')
```

6. Rename Columns Question: Rename the column Age to Years.

CODE:

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}

df = df.rename(columns={'Age': 'Years'})

print(df,'\n')
```

7. Sorting Values Question: Sort the DataFrame by Salary in descending order.

CODE:

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}

df_sorted = df.sort_values(by="Salary", ascending=False)

print(df,'\n')
```

8. Grouping and Aggregation Question: Create a new DataFrame: python CopyEdit data = { 'Department': ['HR', 'IT', 'HR', 'IT'], 'Employee': ['Alice', 'Bob', 'Charlie', 'David'], 'Salary': [50000, 60000, 55000, 65000] } Group by department and find the average salary.

CODE:

```
data = {
```

```

'Department': ['HR', 'IT', 'HR', 'IT'],
'Employee': ['Alice', 'Bob', 'Charlie', 'David'],
'Salary': [50000, 60000, 55000, 65000]
}
df = pd.DataFrame(data)
average_salary = df.groupby('Department')['Salary'].mean().reset_index()
print(average_salary)

```

9. Merge Two DataFrames Question: Given two DataFrames: python CopyEdit
df1 = pd.DataFrame({ 'ID': [1, 2, 3], 'Name': ['Alice', 'Bob', 'Charlie'] }) df2 =
pd.DataFrame({ 'ID': [1, 2, 4], 'Score': [85, 90, 88] })

CODE:

```

import pandas as pd
df1 = pd.DataFrame({ 'ID': [1, 2, 3], 'Name': ['Alice', 'Bob', 'Charlie'] })
df2 = pd.DataFrame({ 'ID': [1, 2, 4], 'Score': [85, 90, 88] })
merged_df = pd.merge(df1, df2, on='ID')
print(merged_df)

```

10. Handling Missing Values Question: Create a DataFrame with some missing values and fill the missing data with appropriate default values.

CODE:

```

import numpy as np
import pandas as pd

data = {
    "Employee": ["Alice", "Bob", "Charlie", "David"],
    "Department": ["HR", np.nan, "IT", "Sales"],
    "Salary": [50000, 60000, np.nan, 55000],
    "Bonus": [2000, np.nan, 1500, np.nan],
}

```

```
df = pd.DataFrame(data)
```

```
print(df)
```

```
print("\n")
```

```
default_values = {
```

```
    "Department": "Unknown",
```

```
    "Salary": df["Salary"].mean(),
```

```
    "Bonus": 0,
```

```
}
```

```
df_filled = df.fillna(value=default_values)
```

```
print(df_filled)
```